



Human Technopole

Nextflow zero2hero

27 January 2026

Nextflow zero2hero

Nextflow Basic Configuration



Summary

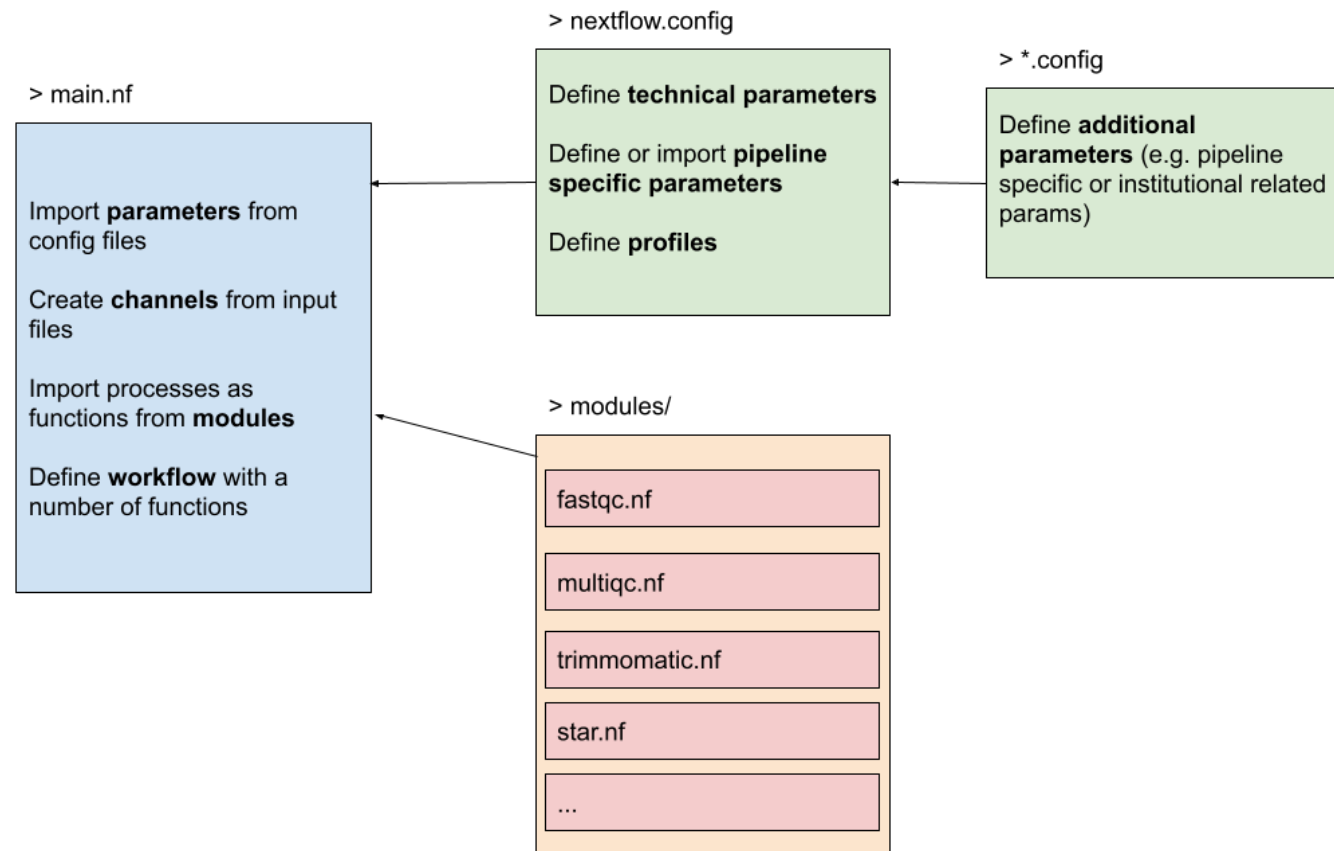
- Configuration Goals
- Set Environment (Container / Conda)
- Define Resources for a Process
- Executors
- Input Params (JSON/YAML)

Nextflow configuration

Configuration Goals

The main goal of this section is to explore how to manage a Nextflow pipeline's configuration to customize its behavior, optimize resource usage, and adapt it to different environments **without altering the workflow code itself**.

This is primarily achieved using the `nextflow.config` file, and settings specified here can be overridden at runtime using command-line parameters or other configuration files.



Nextflow configuration

Set environment (container / conda)

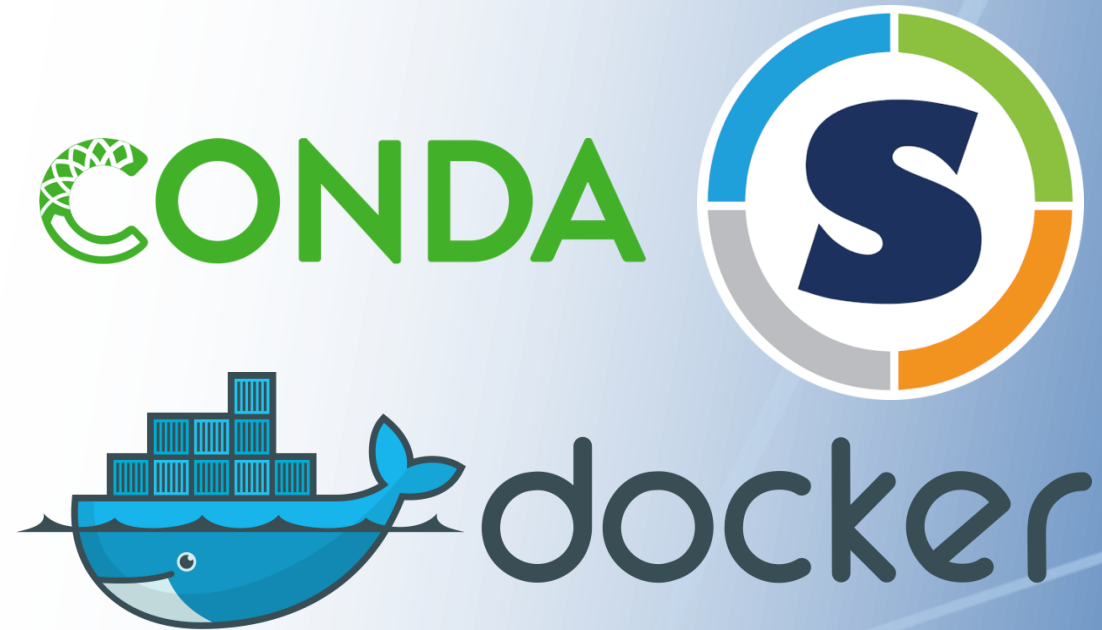
The first step in adapting the workflow configuration is determining where the necessary software packages originate.

Nextflow supports various container technologies, such as **Singularity/Apptainer** (often used on HPC), and package managers like **Conda**.

When using Conda, the package URI must be specified using the **conda directive** within the process definition.

It is possible to mix and match technologies, configuring some processes for Docker and others for Conda.

If both are available for a given process, Nextflow will prioritize containers.



Nextflow configuration

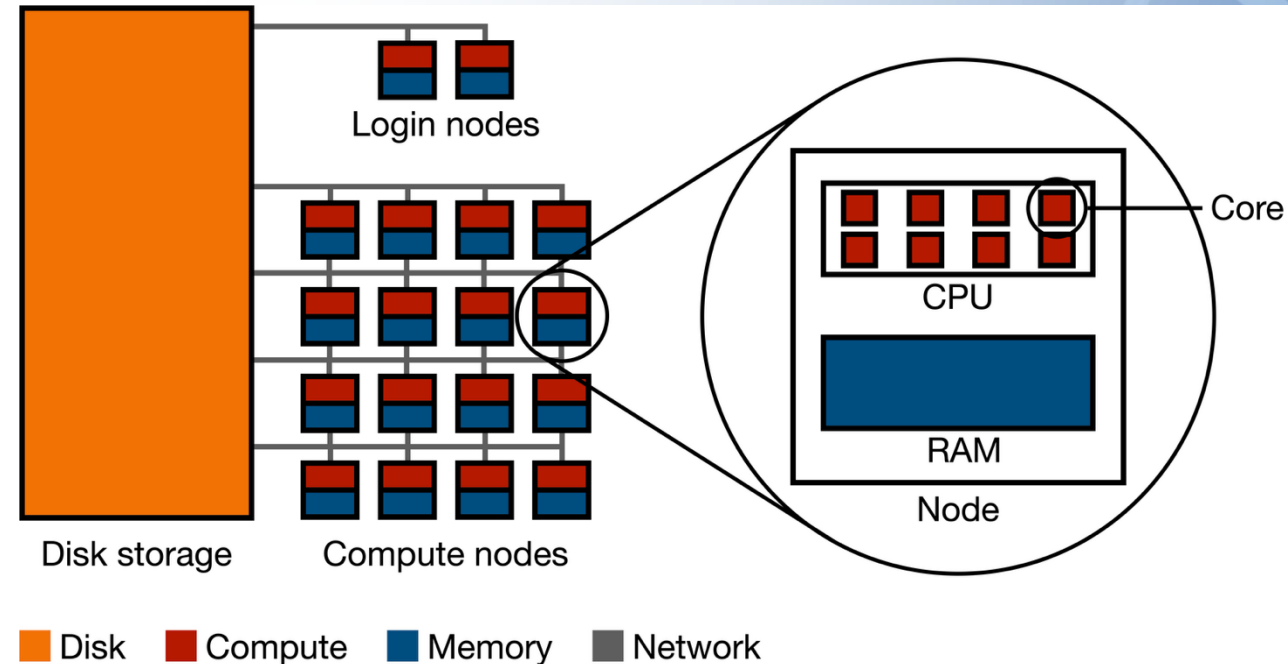
Define resources for a process

High-performance computing systems often require users to specify requests and limitations for compute resources like memory and CPUs.

By default, Nextflow uses 1 CPU and 2GB of memory for each process.

These values can be modified in the `nextflow.config` file for all processes, or specifically for individual processes.

Defining these resources allows the pipeline to right-size allocations based on actual data, which is useful when processes have differing requirements.



Nextflow configuration

Executors

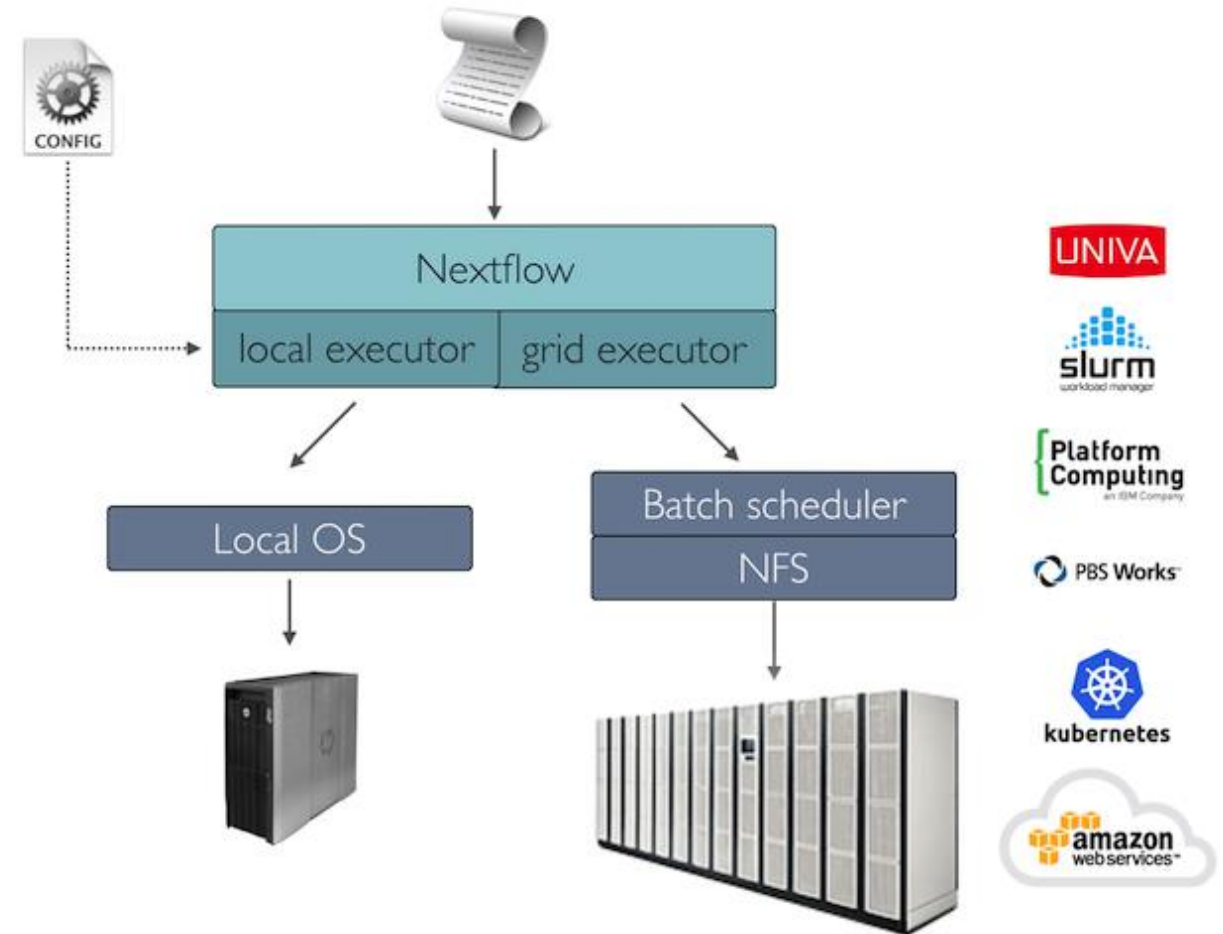
The executor specifies the backend that Nextflow should use to perform the work.

By default, the **local executor** is used, which runs tasks on the machine where Nextflow is executing.

The local executor is limited to that single machine.

For very large workloads, Nextflow supports many **execution backends**, including HPC schedulers like **Slurm**, LSF, and PBS, as well as cloud backends like AWS Batch and Kubernetes.

Nextflow simplifies execution parameter definitions by using a standardized syntax (like cpus, memory, and queue) and then translating these settings into the appropriate backend-specific scripts at runtime.



Nextflow configuration

Use JSON/YAML for input params

Managing workflow parameters is crucial for reproducibility.

While simple workflows can use command-line arguments, many real-world workflows require numerous parameters.

Nextflow allows users to specify parameter values via external files in either **YAML** or **JSON** format, which makes it convenient to manage and distribute sets of parameters.

Parameters can be passed to the run command using the **-params-file <filename>** option. Default parameter values can also be stored in the nextflow.config file within a params block.

```
! params.yaml ×
Volumes > nfdata-omics > projects > WGS_Faletti > ! params.yaml
1
2   input:  "/facility/nfdata-omics/projects/WGS_Faletti/samplesheet.csv"
3
4   outdir: "/facility/nfdata-omics/projects/WGS_Faletti/results"
5
6   genome: "GATK.GRCh38"
7
8   aligner : "bwa-mem2"
9
10  trim_fastq: TRUE
11
12  save_trimmed: TRUE
13
14  save_mapped: TRUE
15
16  tools: "ascart,strelka,mulect2,deepvariant"
17
18  split_fastq : 10000000
19
20
```

Nextflow configuration Practices



Now let's see how this works in practice

Thank you.

Fondazione Human Technopole

Palazzo Italia

Viale Rita Levi-Montalcini, 1 - 20157
Milan, Italy

humantechnopole.it



humantechnopole.it